faculdade de tecnologia
departamento de engenharia elétrica

Service-oriented analysis and design

Prof. Ricardo Staciarini Puttini, Dr.

What is SOA?

Common Misconceptions

- SOA is a thing
- SOA is a middleware technology (i.e., WS-*)
- SOA is a new name for integration
- SOA is about orchestrating business processes
- SOA is all about reuse

SOA is none of these things

SOA is something you do, not something you buy

What is SOA?

The Big Picture

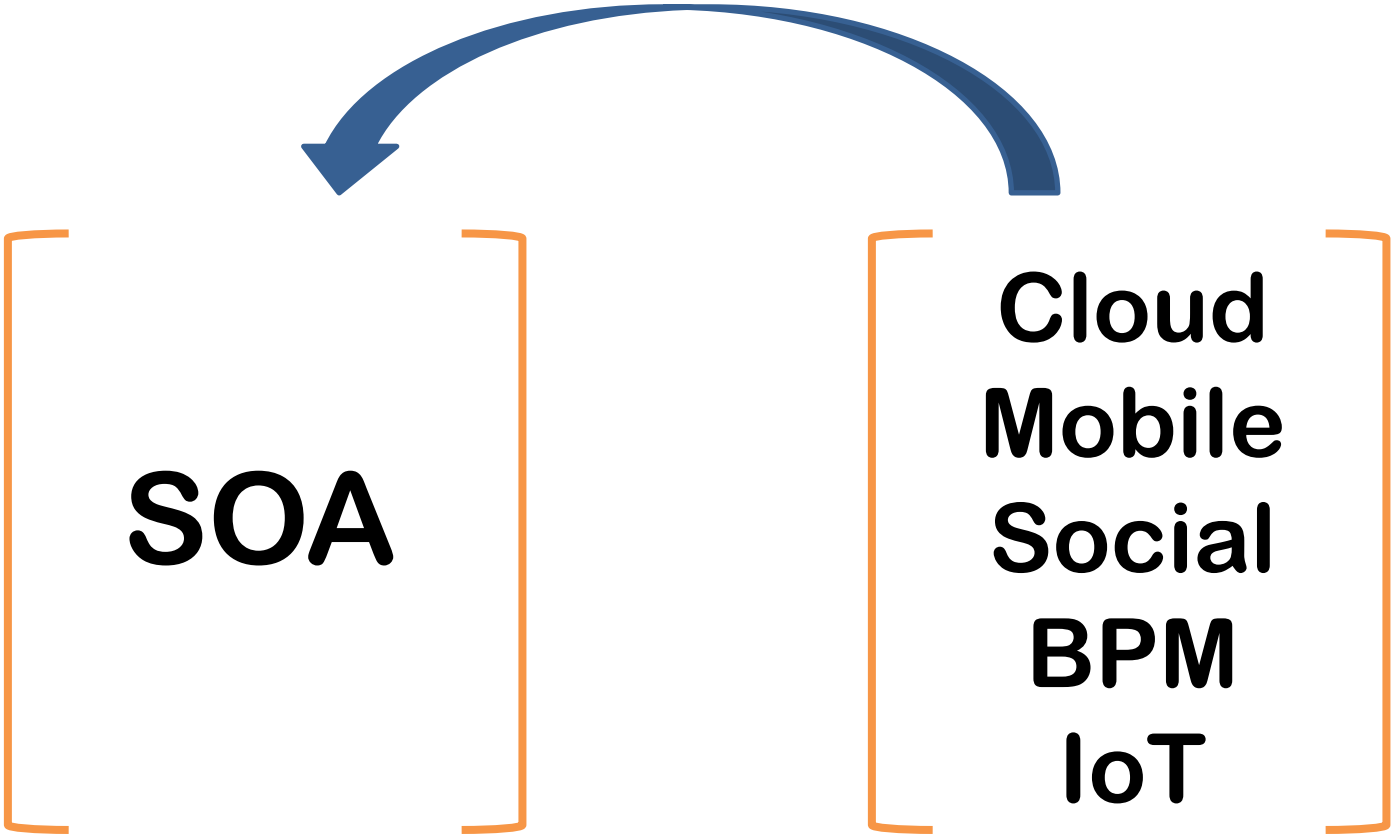
"A Service-Oriented Architecture (SOA) is a set of principles and methodologies for designing and developing software in the form of interoperable services. These services are well-defined business functionalities that are built as software components that can be reused for different purposes."

-- Wikipedia

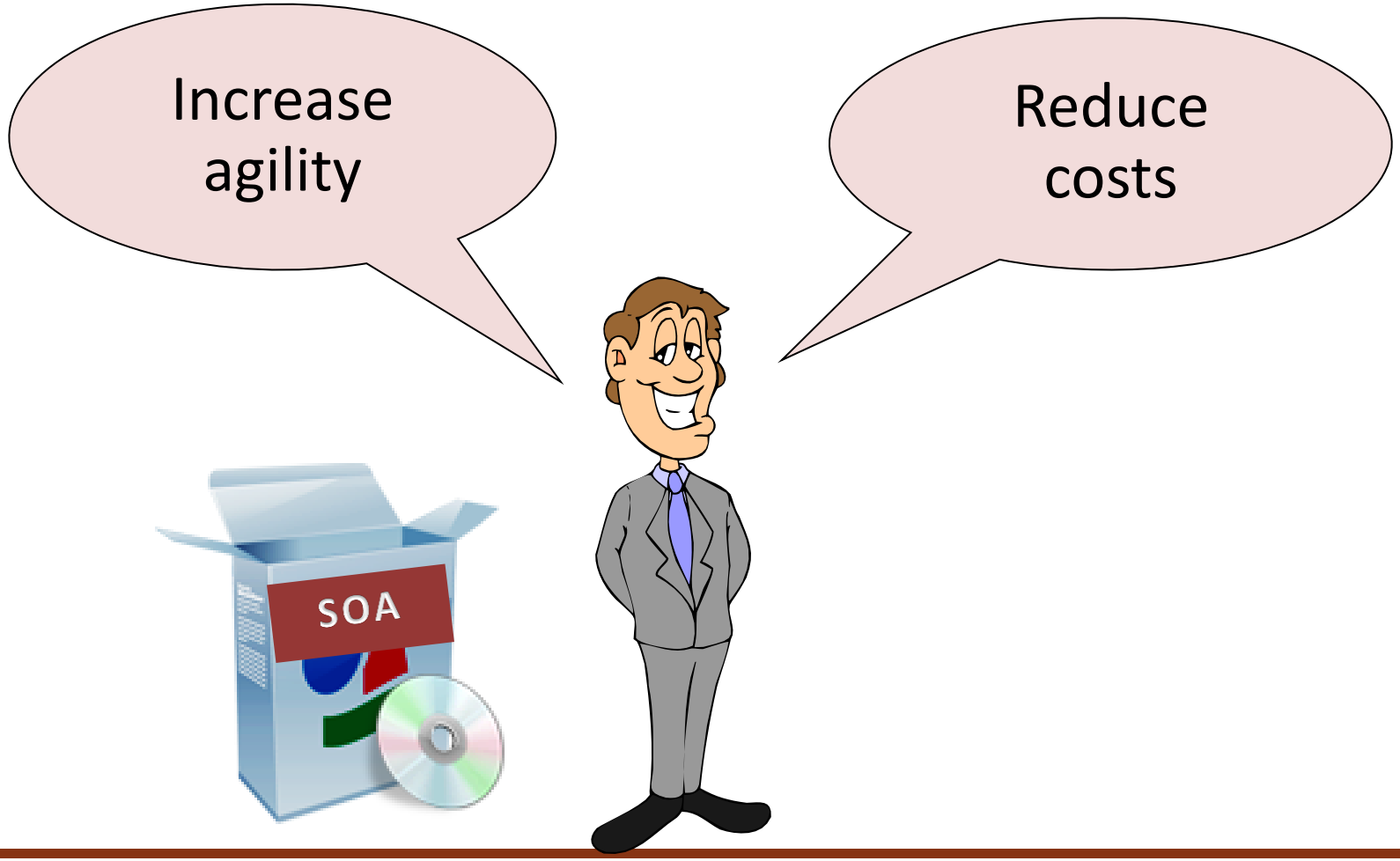
Comparing SOA

SOA	Design paradigm Service-centric, not application-centric Reduces replication and duplication
Integration	Bridges application silos Increases replication and duplication
Orchestration	Application development methodology Composes and consumes services Requires pre-existing portfolio of services
Middleware	Communications technology Used to communicate between endpoints

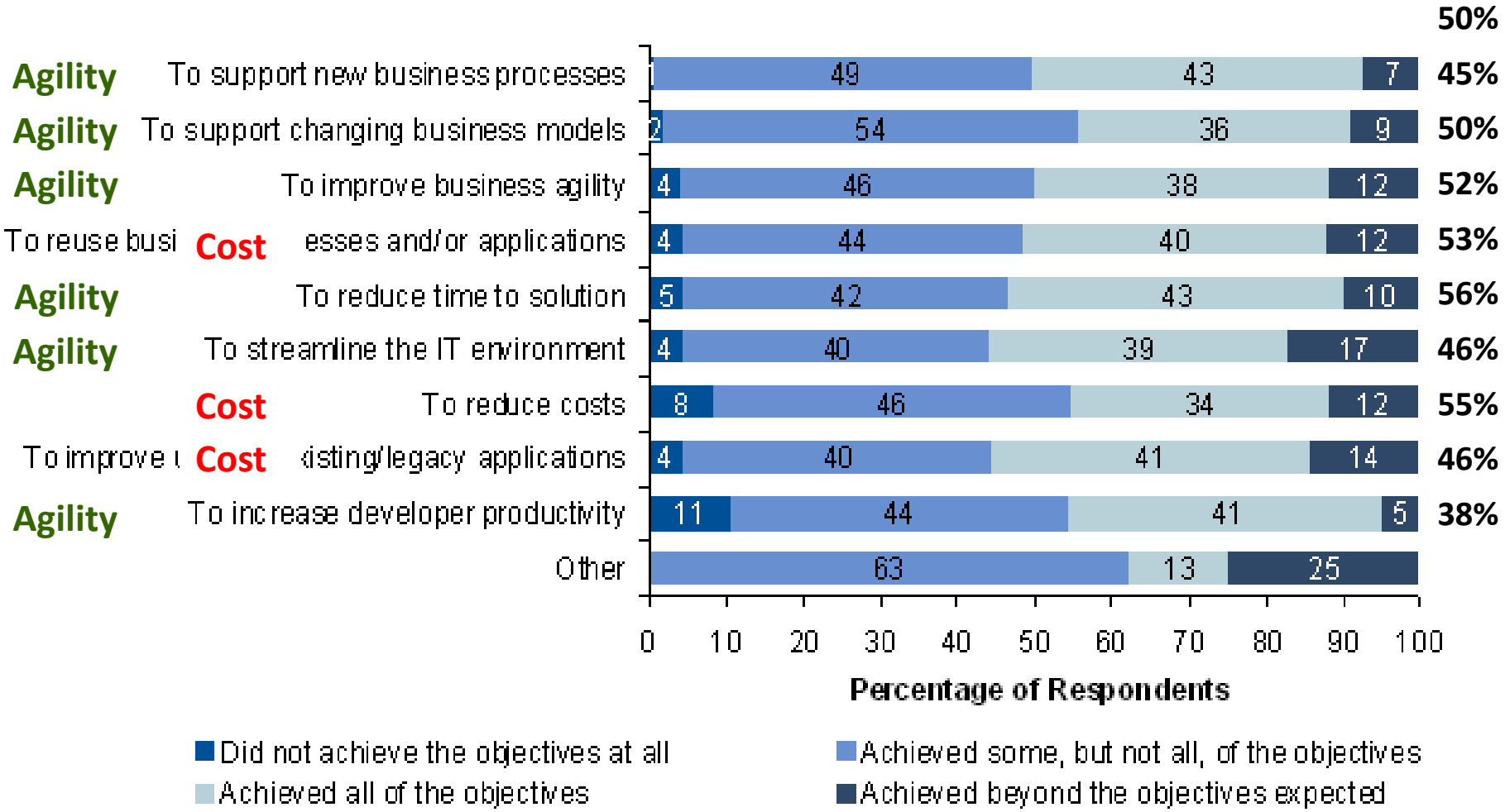
SOA is a Prerequisite



SOA Business Benefits

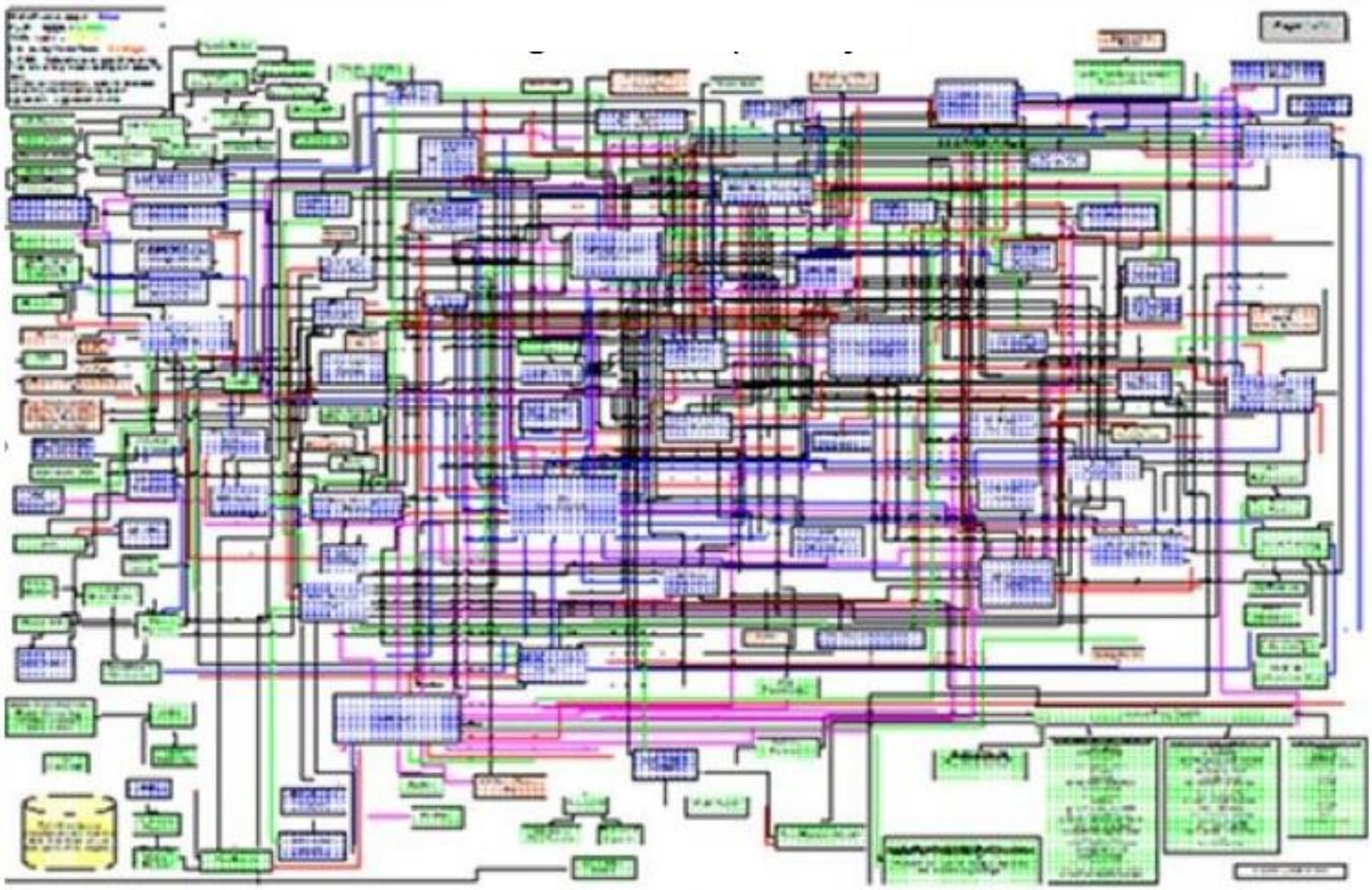


Survey of SOA Objectives



* **User Survey Analysis: What's Really Happening With SOA in Tough Times?** – Susan Tan , Gartner

What impedes agility
What increase costs



Architecture Impacts Business Value

Multiplicity	• Support new uses • Support new business models
Versatility	• Adapt to changing requirements • Reuse for different processes
Usability	• Enable easy access • The right information at the right time
Maintainability	• Enable easy modification • Readability of code
Robustness	• Performance and scalability • Reliability, availability, recoverability

Translate Biz Goals into Technical Goals

Business Goals and Objectives	
Increase agility	Reduce costs

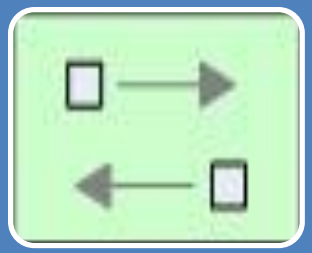
Technical Goals and Objectives		
Increase usability	Improve maintainability	Reduce redundancy

Defining SOA

Service orientation is a design paradigm.

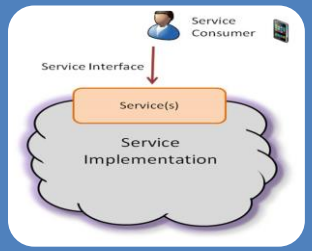
SOA results from applying the SO paradigm.

SOA is a Set of Design Principles



Loose coupling

- Decoupled Transactions
- Mediated Interactions



Encapsulation

- Accessibility
- Granularity

Clean separation of concerns

- Business Logic from Infrastructure
- Interface from Implementation

The Principles Yield SOA

Interoperable

- Easy to use
- Well-defined interfaces

Modular

- Easy to maintain
- Separation of concerns

Other SOA Characteristics

Distributable

- Easy to deploy or consume
- Distribution is hidden behind the interface

Discoverable

- Easy to find
- Explicit policies, constraints, and contracts

Swappable

- Easy to upgrade or replace
- Based on shared contracts

Shareable

- Reduces redundancy; improves consistency
- Multiple consumers share a single implementation

Services Enable Agility

Integration

- Application integration
- Virtual data warehousing
- Mobile and social applications

Service composition

- New application development
- Leveraging existing assets
- Business process orchestration

Citizen development

- Mashup tools
- Workflow systems
- Spreadsheets and business intelligence tools

Part I: Fundamental SOA

1 Introduction to SOA
2 SOA Design Principles and Patterns
3 SOA Development Methodology
4 SOA Platform Architecture

Design Principles

- Design principles describe fundamental ideas and accepted practices that influence system design.
- Application of principles yields desired characteristics.

Design Patterns

- A proven solution to a common problem within a given context
- Similar to a best practice
- Often implements a paradigm's design principles

Models

- *Modeling* is the process of applying design principles and patterns in context
- *Models* are the artifacts resulting from modeling:
  - Rich descriptions of system capabilities, components, interfaces, policies, dependencies, agreements, etc.

Service-Orientation Design Principles (from T. Erl)

- Standardized Service Contract
- Service Loose Coupling
- Service Abstraction
- Service Reusability
- Service Autonomy
- Service Statelessness
- Service Discoverability
- Service Composability

Standardized Service Contract

"Services within the same service inventory are in compliance with the same contract design standards."

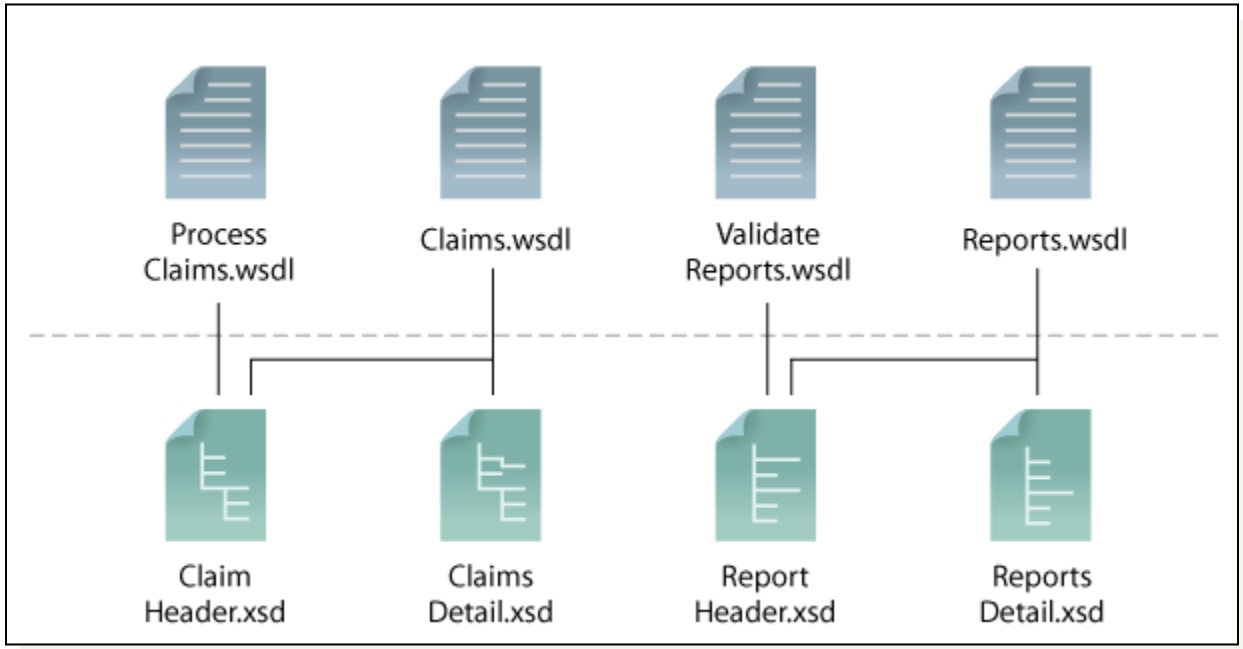
Standardized Service Contract

- Services express their purpose and capabilities via a standardized service contract.
- This places a great deal of emphasis on the design of service contracts and advocates the “contract first” design process.
- Its purpose is to ensure that the manner in which services express functionality and represent data types is kept in relative alignment.
- There are two types of contract standardization:
  - Data Representation (or Data Model) Standardization
  - Functional Expression Standardization

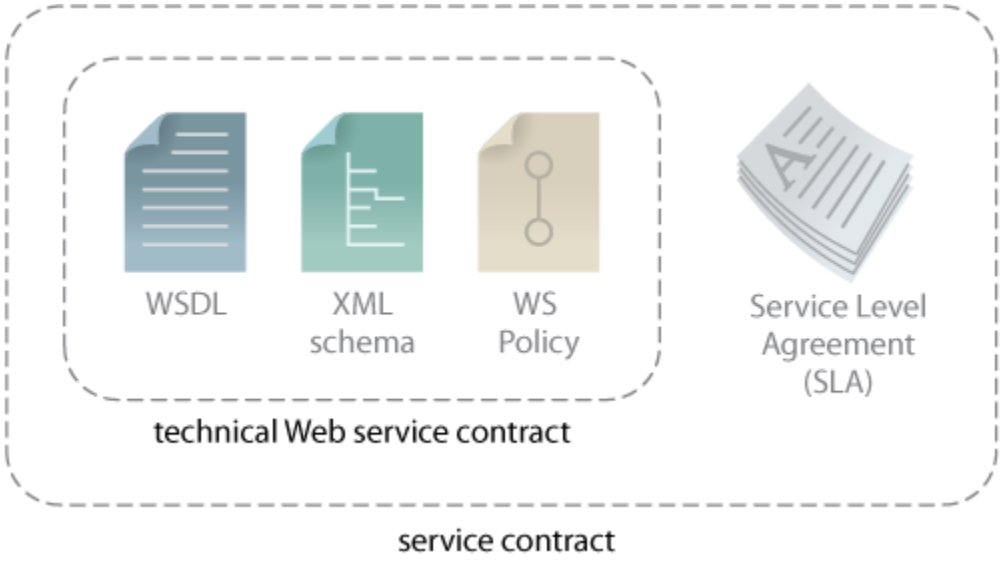
Standardized Service Contract

Data model standardization results in WSDL definitions that share common XML schemas.

These XML schemas define the structure of the input and/or output data for each service capability.



Standardized Service Contract



The above figure shows Web service contract documents affected by this design principle.

Standardized Service Contract

Examples of functional expression conventions:

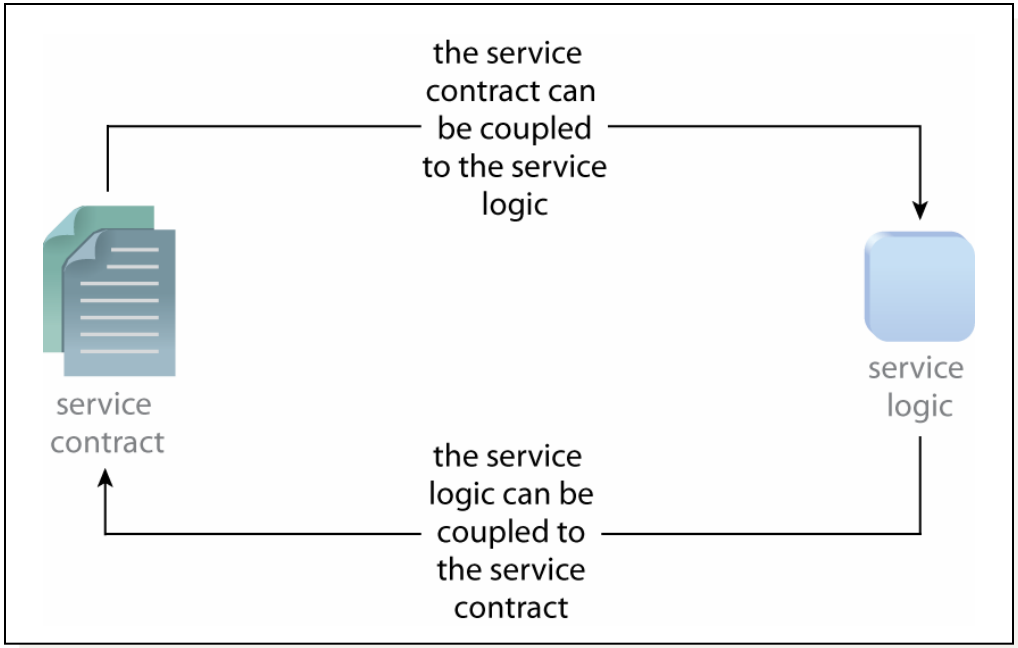
- Entity services will be named in accordance with their corresponding business entities.
- The names of task services will be based on the business process the service is responsible for automating.
- Service capability names will include a verb followed by a noun.
- The service capability name will not repeat the name of the service.

Service Loose Coupling

"Service contracts impose low consumer coupling requirements and are themselves decoupled from their surrounding environment."

Service Loose Coupling

- Coupling refers to a connection between two things.
- The extent of the coupling refers to the extent of dependency between the two things.
- The primary focus of this principle is on the relationship between the service contract and whatever the service encapsulates.

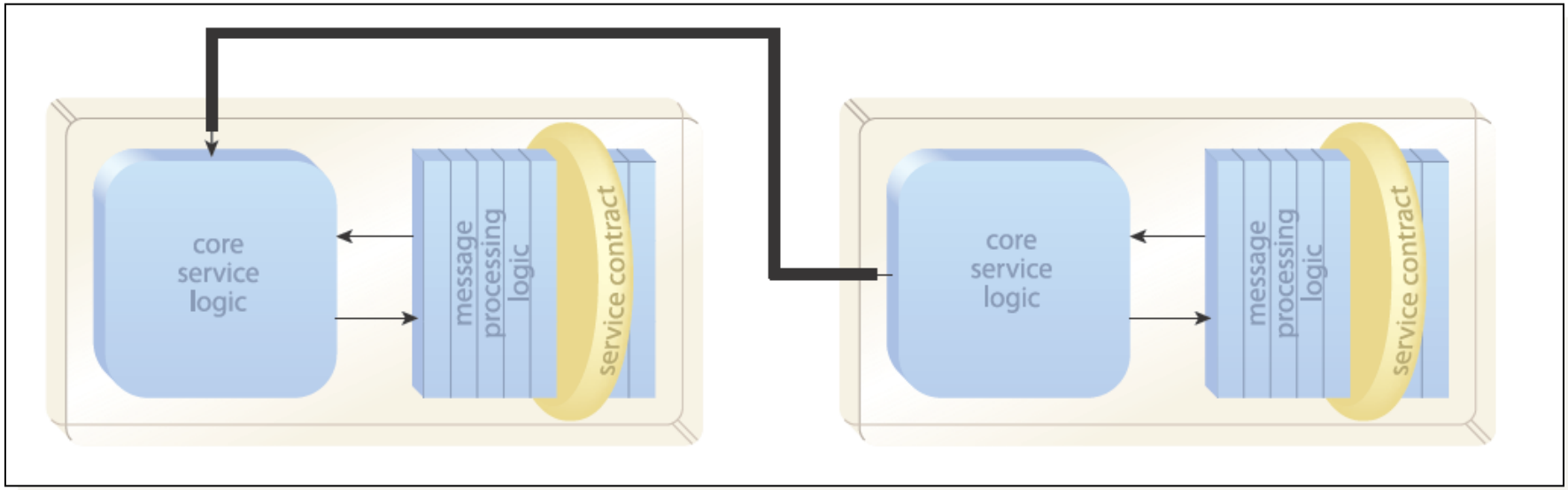


Service Loose Coupling

- A core design consideration when establishing consumer-service relationships is whether or not the service consumer actually couples to the published service contract.
- This principle (and service-orientation in general) strongly emphasizes that service consumers only couple to a service's contract, which is known as Consumer-to-Contract coupling.
- The application of the Service Loose Coupling principle primarily focuses on variations of the consumer-to-contract coupling type.

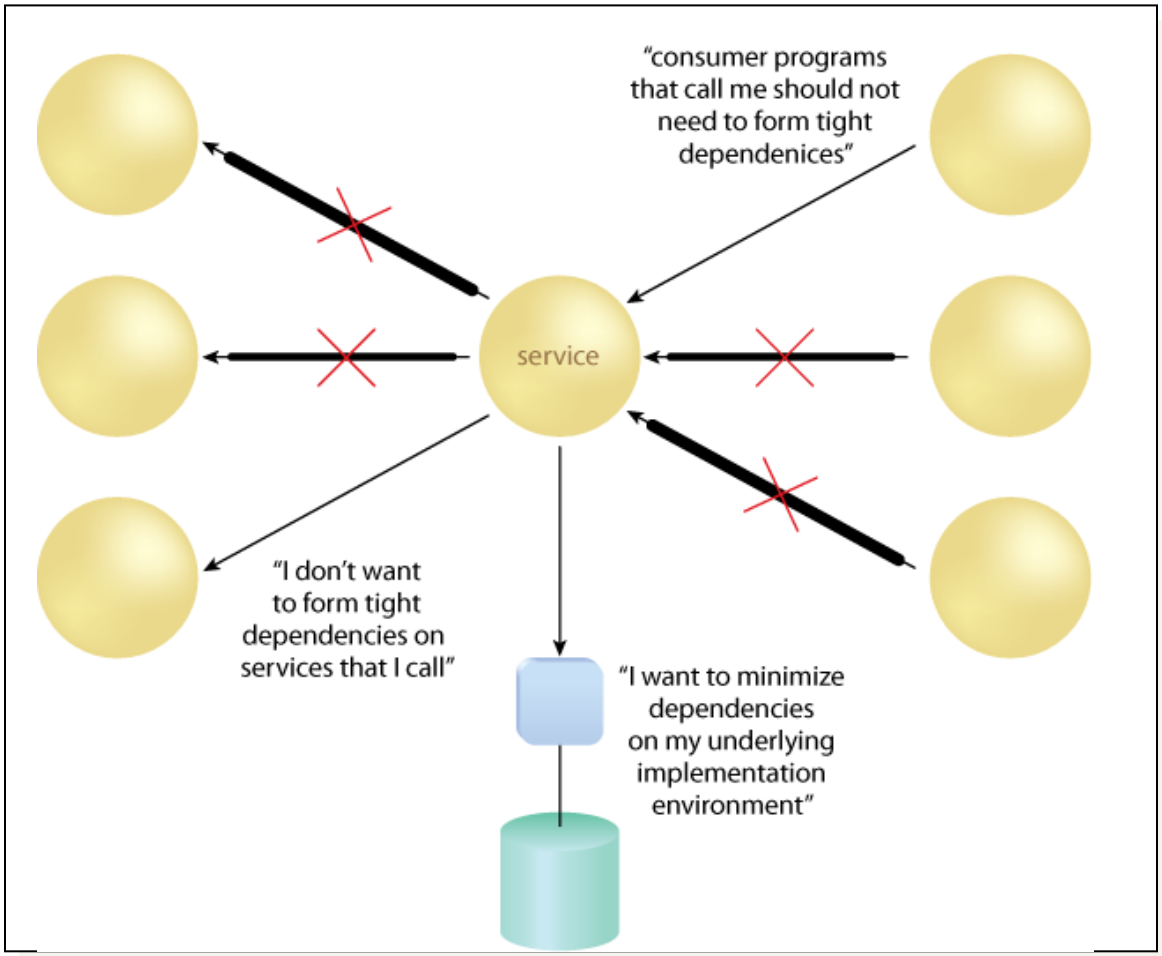
Service Loose Coupling

The fundamental negative form of consumer coupling occurs when the consumer program is designed to bypass the service contract in order to access parts of the service's underlying implementation directly. This negative coupling type is referred to as Consumer-to-Implementation coupling.



Service Loose Coupling

This principle emphasizes reducing ("loosening") dependencies within and outside of service boundaries.



Service Loose Coupling

When designing services, the following coupling types need to be taken into consideration in order to achieve the appropriate level of overall coupling:

A. Logic-to-Contract (positive)

B. Contract-to-Logic (negative)

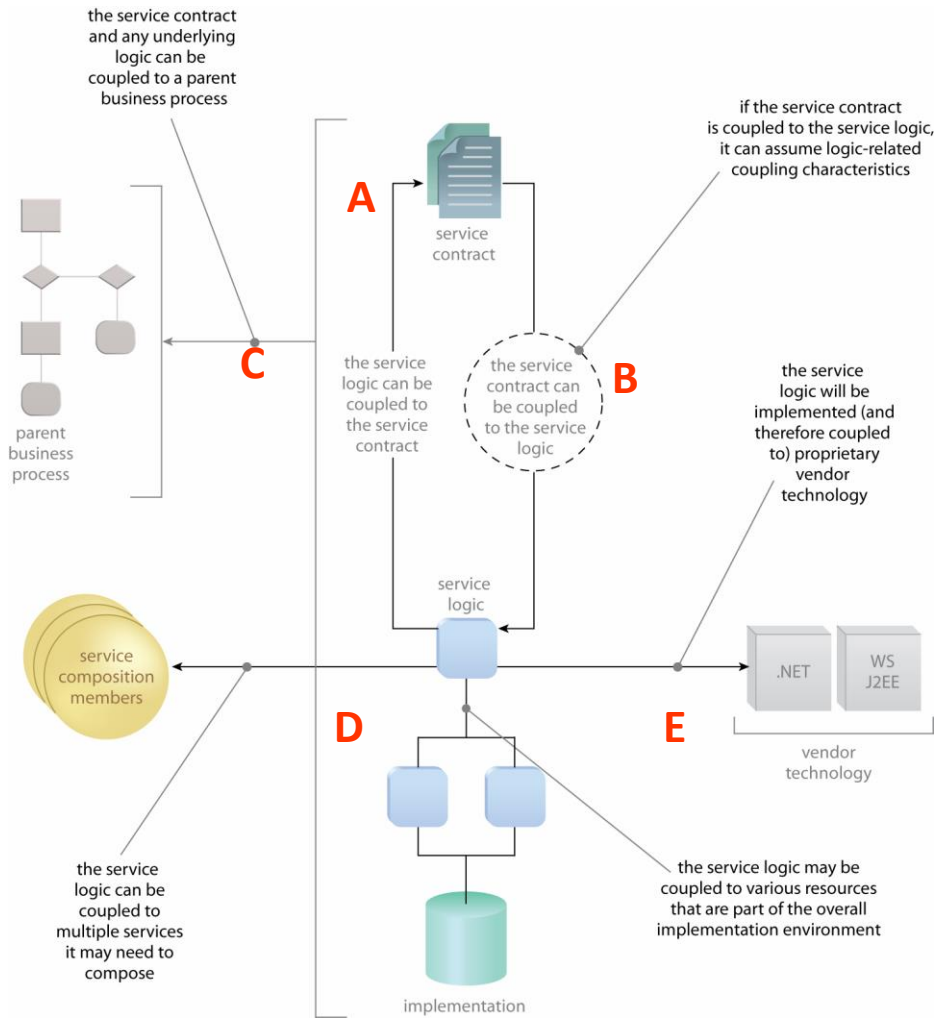
C. Contract-to-Functional (negative)

D. Contract-to-Implementation (negative)

E. Contract-to-Technology (negative)

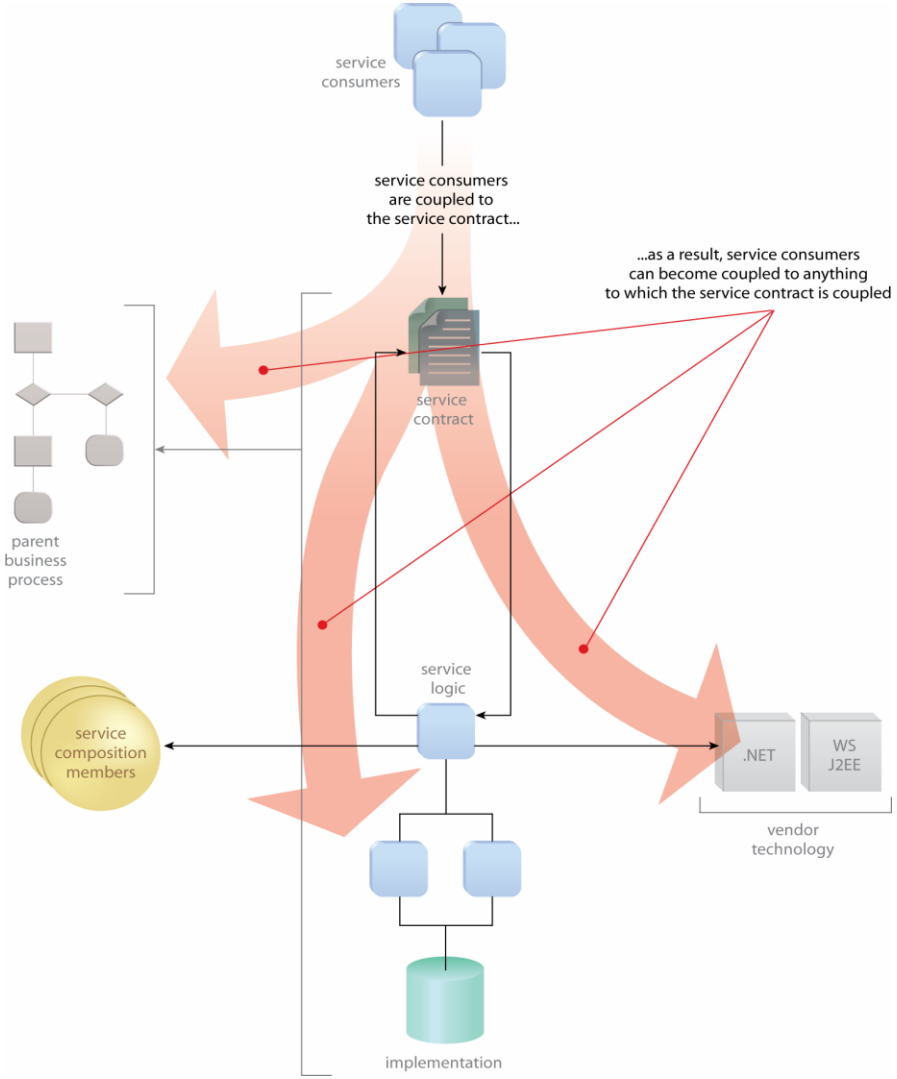
Negative contract coupling types are avoided in order to maximize the governance independence of the service contract. A fully decoupled service contract can avoid imposing negative coupling types upon service consumers.

There are several distinct forms of dependencies (types of coupling) a service can have, all of which relate back to the service contract.



Ultimately, any undesirable forms of coupling allowed into the service contract design end up being imposed on and proliferated through all consumers of the service.

This causes serious problems when agnostic services are highly reused.



Service Abstraction

"Service contracts only contain essential information and information about services is limited to what is published in service contracts."

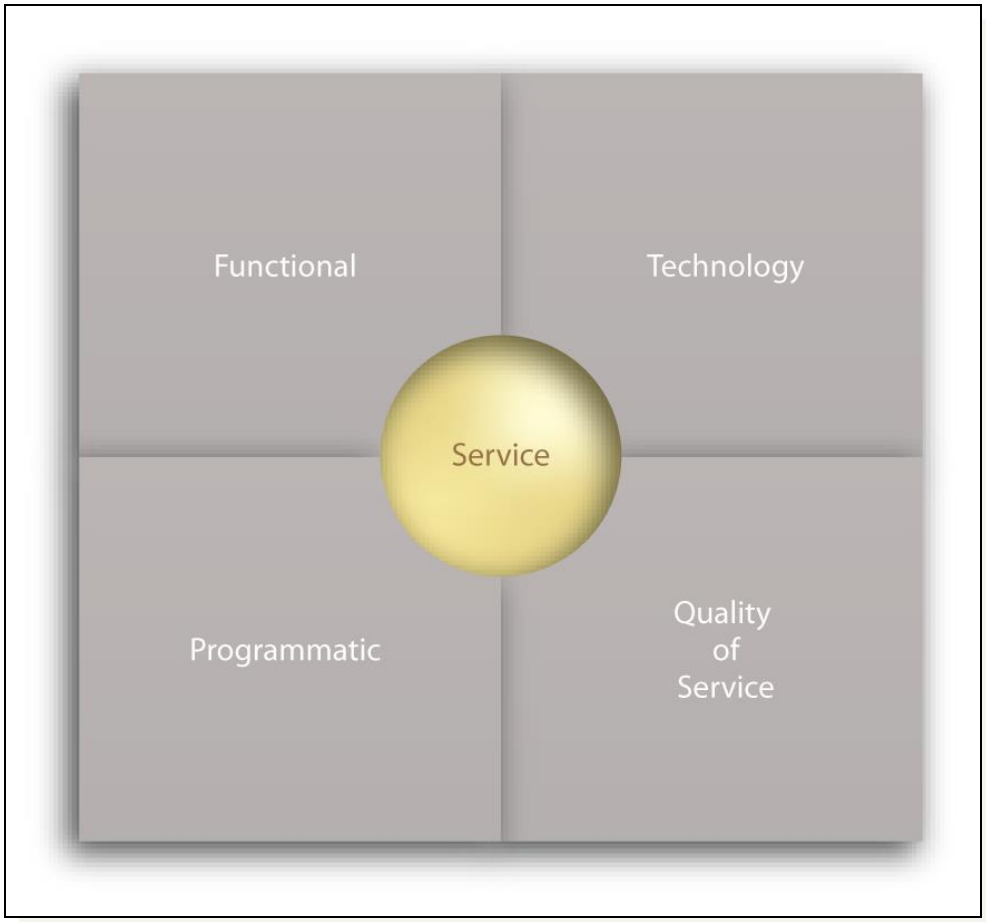
Service Abstraction

- On a fundamental level, this principle emphasizes the need to hide as much of the underlying details of a service as possible.
- Information about a service is thereby limited to what is published in the service contract.
- However, service abstraction influences the content of the service contract as well.
- This principle directly supports Service Loose Coupling.

Service Abstraction

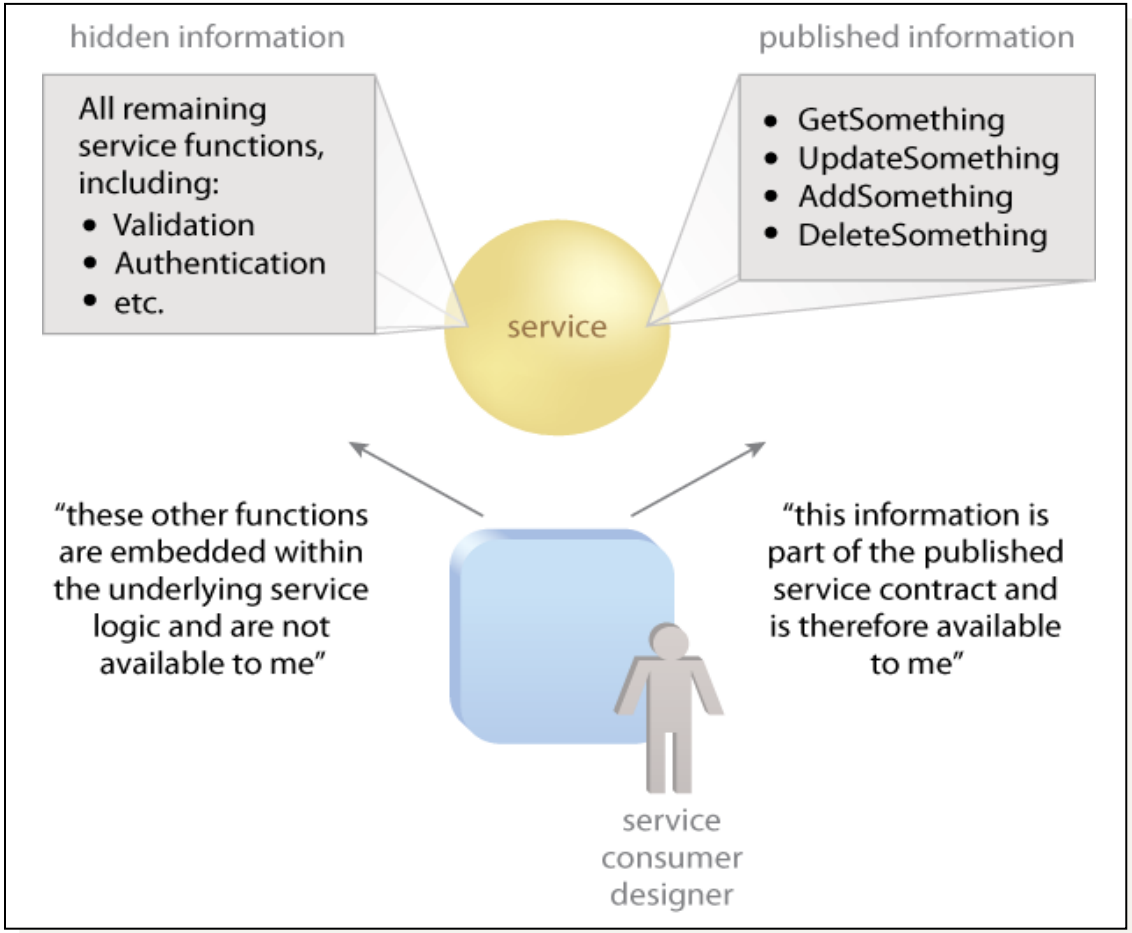
There are four meta abstraction types:

- functional
- technology
- programmatic
- quality-of-service



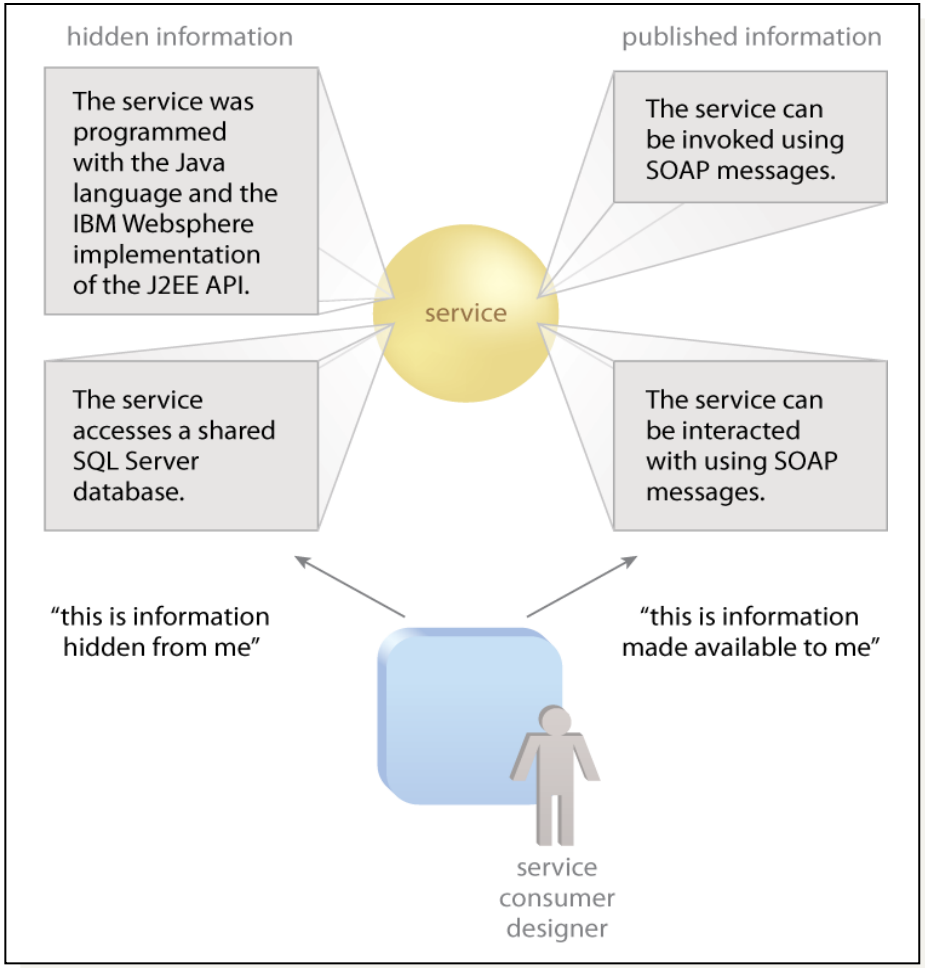
Service Abstraction

Functional abstraction relates to meta data that describes what the service is capable of.



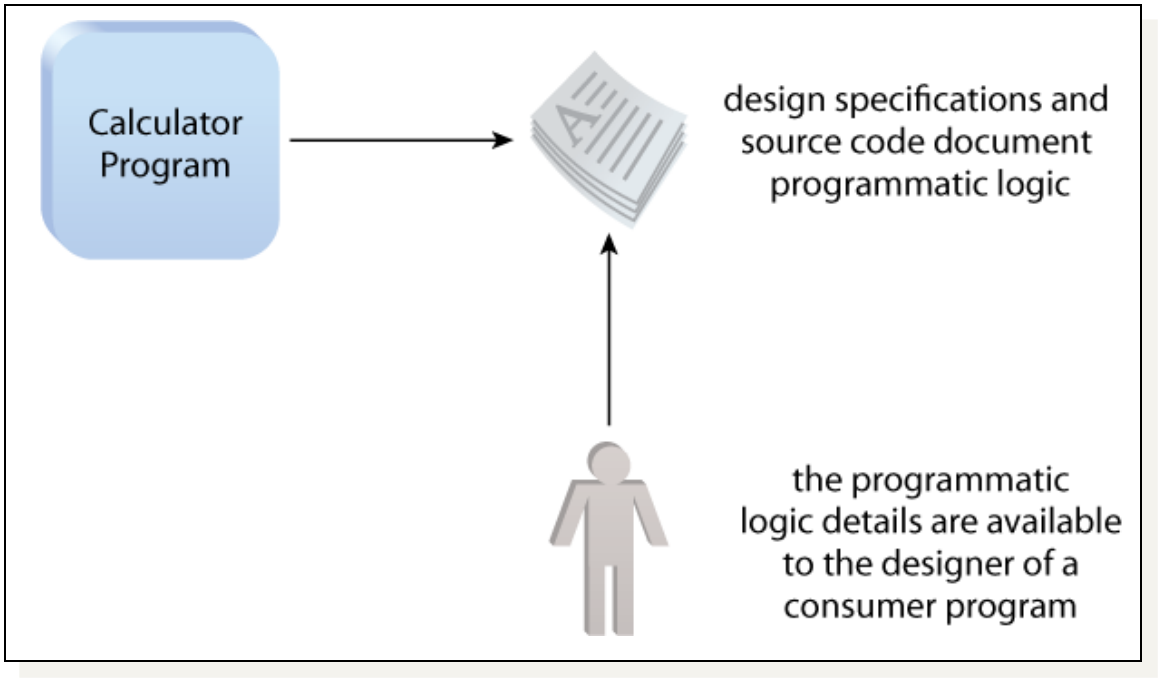
Service Abstraction

Technology abstraction relates to meta data that describes the technical implementation of the underlying service logic.



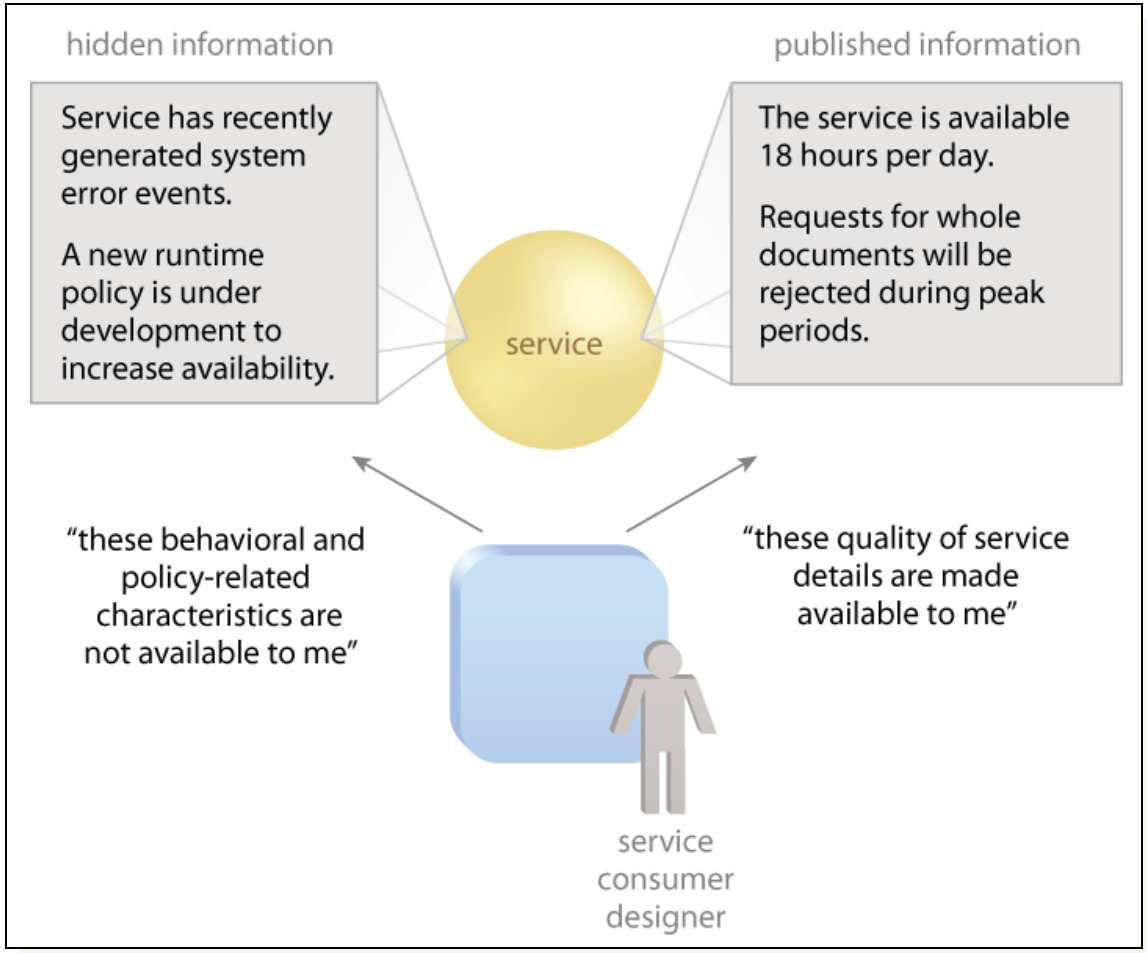
Service Abstraction

Programmatic logic abstraction relates to meta data that describes how the service carries out its capabilities.



Service Abstraction

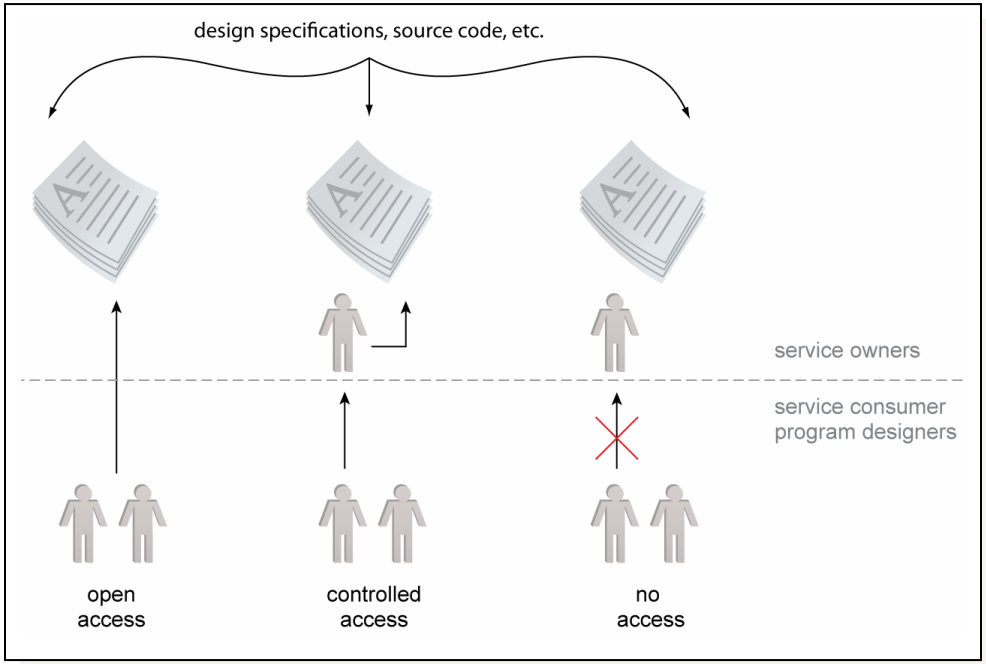
Quality-of-service abstraction relates to meta data that describes service behavior, limitations, and interaction requirements.



Service Abstraction

Applying this principle may involve the need to introduce access control measures for service design specifications and source code.

This protects both the service and its consumers from undesirable forms of coupling.



Service Reusability

"Services contain and express agnostic logic and can be positioned as reusable enterprise resources."

Service Reusability

- This principle asks us to identify any potentially reusable logic and make it available within an agnostic service context.

- “Reusability” is a term used to indicate the potential for a software program to be reused, whereas “reuse” is the actual act of reusing the program.

- Never before has reuse been as emphasized as with service-orientation (so much so, that it becomes a core part of the design process).

Service Reusability

The purpose of the design approach is not to guarantee reuse, but to instead apply expert judgment when determining:

- the most suitable *type* of logic, and…

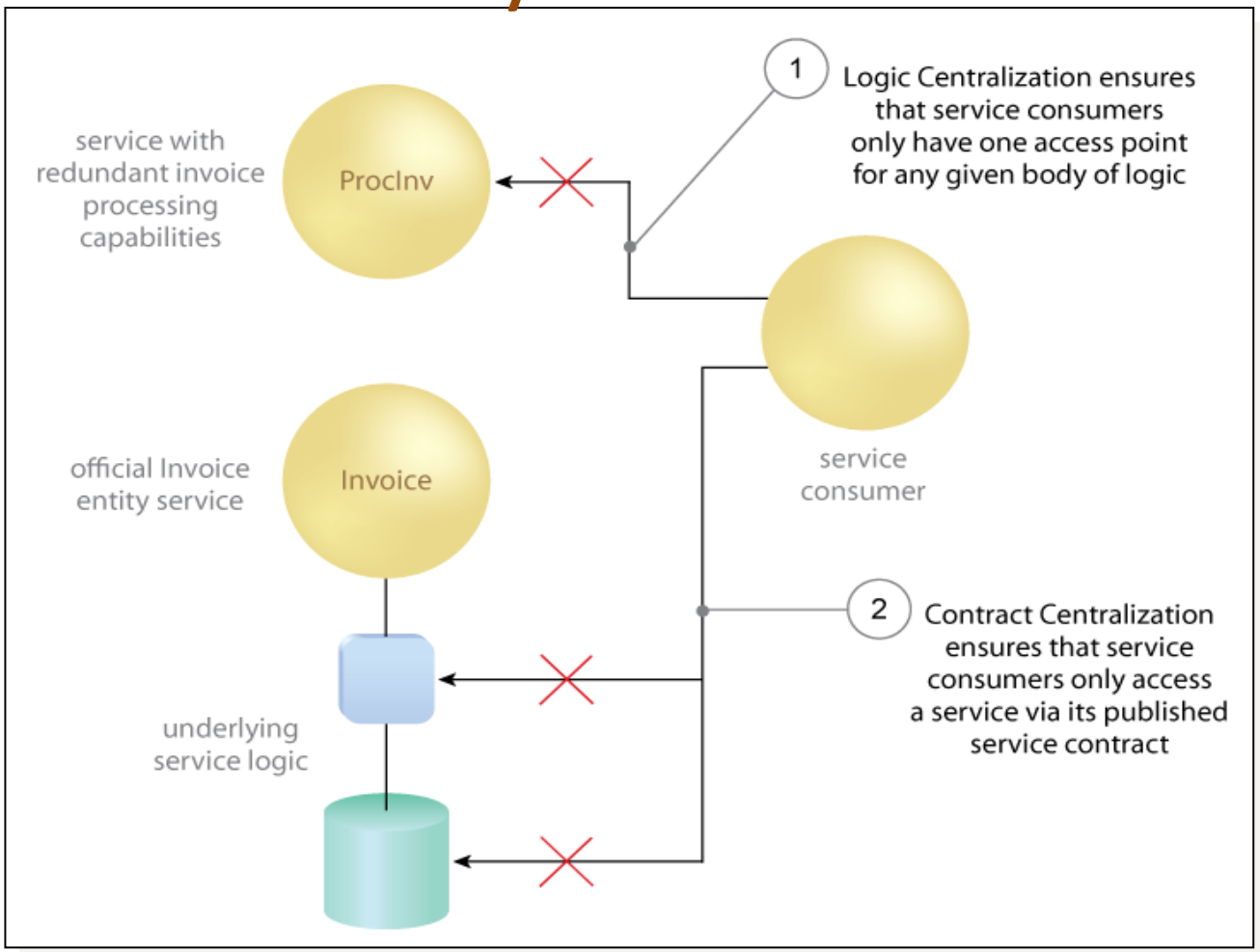
- the most suitable *quantity* of logic

… to be provided by a particular version of a reusable service.

This maximizes the reuse potential and reduces the risk associated with over-equipping a service with excessive features.

Service Reusability

Logic and contract centralization result in a highly standardized and normalized service inventory in full support of maximizing reusability potential and loose consumer coupling.



Service Autonomy

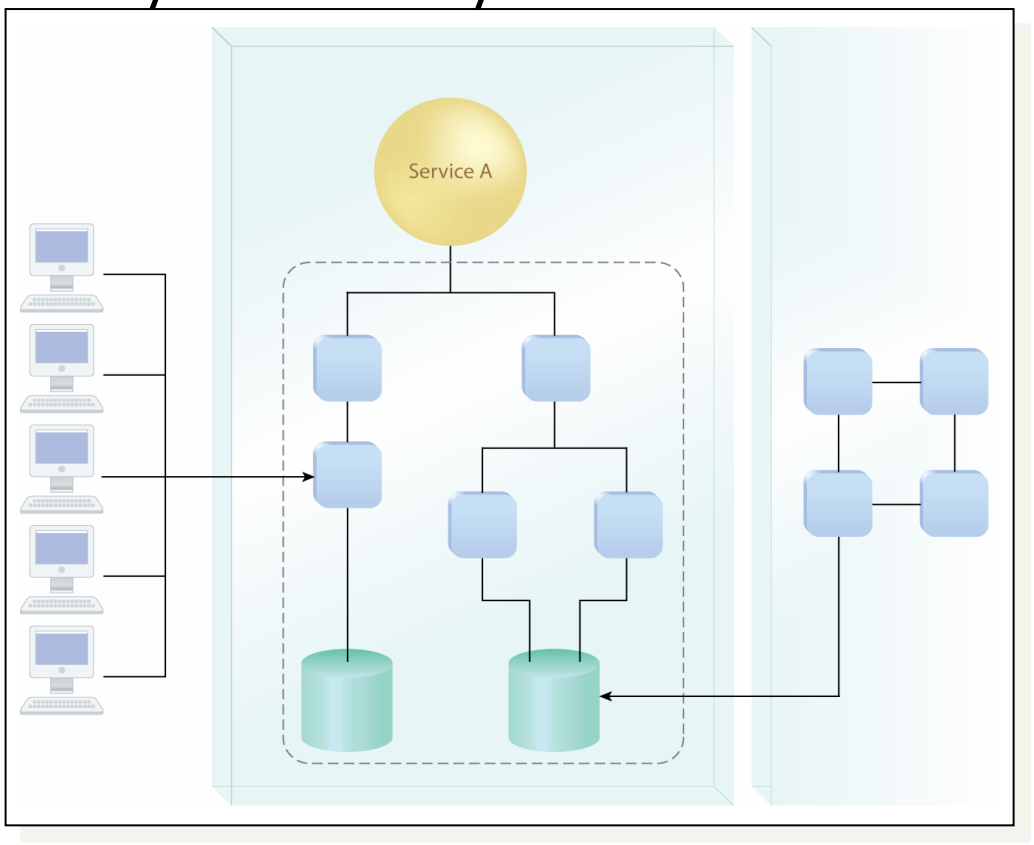
"Services exercise a high level of control over their underlying runtime execution environment."

Service Autonomy

- The principle of autonomy refers to:
  - the level of control we have over a service at design-time (design-time autonomy)
  - the level of control the service has over its execution environment at runtime (runtime autonomy)
- There are different levels of attainable runtime autonomy, depending on the extent of isolation a service implementation is given.

Service Autonomy

- Applying this principle requires us to assess the amount of shared resources a service may need to rely on.
- Any part of a service's environment over which it does not have exclusive control can compromise the predictability of its performance and behavior.

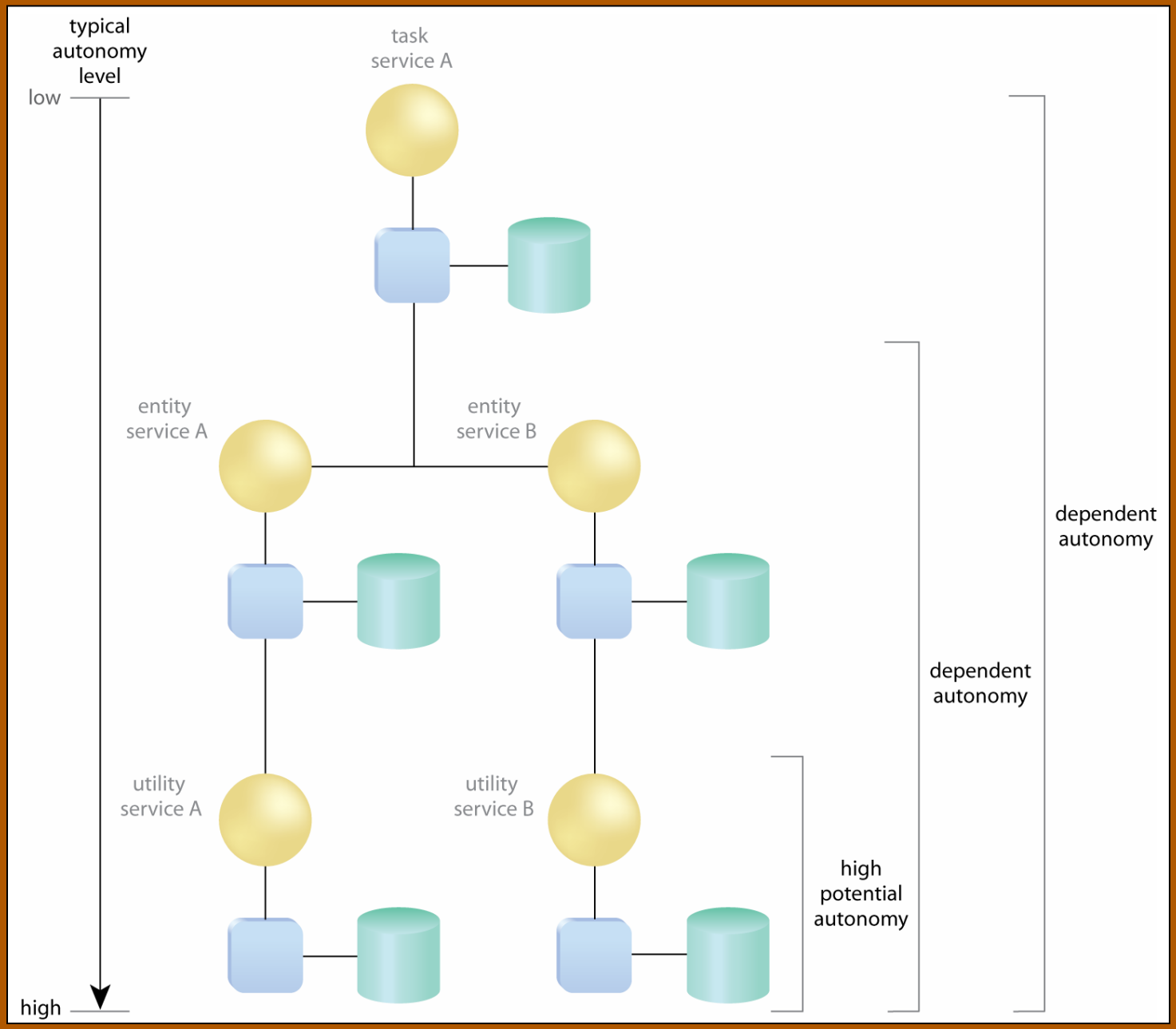


Service Autonomy

- Two primary benefits of raising the level of autonomy within a program are to increase its reliability and behavioral predictability.
- Achieving a higher level of autonomy is especially important for agnostic services, such as those based on the entity and utility service models.
- Autonomy supports the reusability and composability of services.

When composing services, autonomy is automatically lost the further up you go in a composition hierarchy.

It is therefore extra important to make each composition member as autonomous as possible.



Service Statelessness

"Services minimize resource consumption by deferring the management of state information when necessary."

Service Statelessness

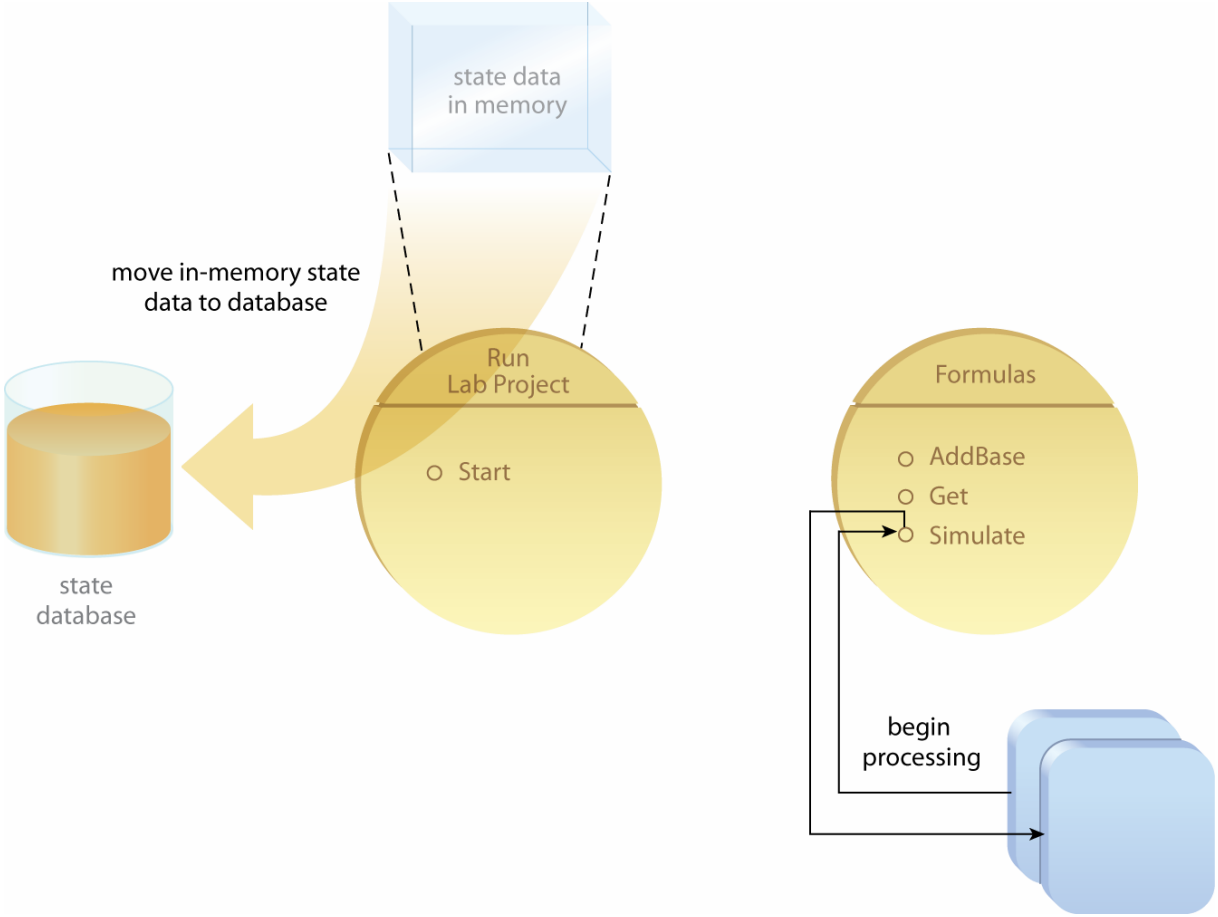
- The management of excessive state information can compromise the availability and scalability of a service.
- Services are therefore ideally designed to remain in a stateless condition whenever appropriate.
- Applying this principle usually involves some form of state deferral.
- Like Service Autonomy, this is another principle that focuses less on the contract and more on the design of the underlying logic.

Service Statelessness

- When automating a particular task, a service is required to process data specific to that task. This is referred to as state data.
- A service can be active but may not be engaged in the processing of state data. In this idle condition, the service is considered to be stateless.
- A service that is actively processing or retaining state data is classified as being stateful.

Service Statelessness

Common state deferral options include databases, utility services, and messages.



Service Discoverability

"Services are supplemented with communicative meta data by which they can be effectively discovered and interpreted."

Service Discoverability

It is desirable for services (especially agnostic services) to be easily located and understood so that they can be appropriately reused and recomposed.

There are four concepts relevant to accomplishing this:

- discovery
- discoverability
- interpretation
- interpretability

Service Discoverability

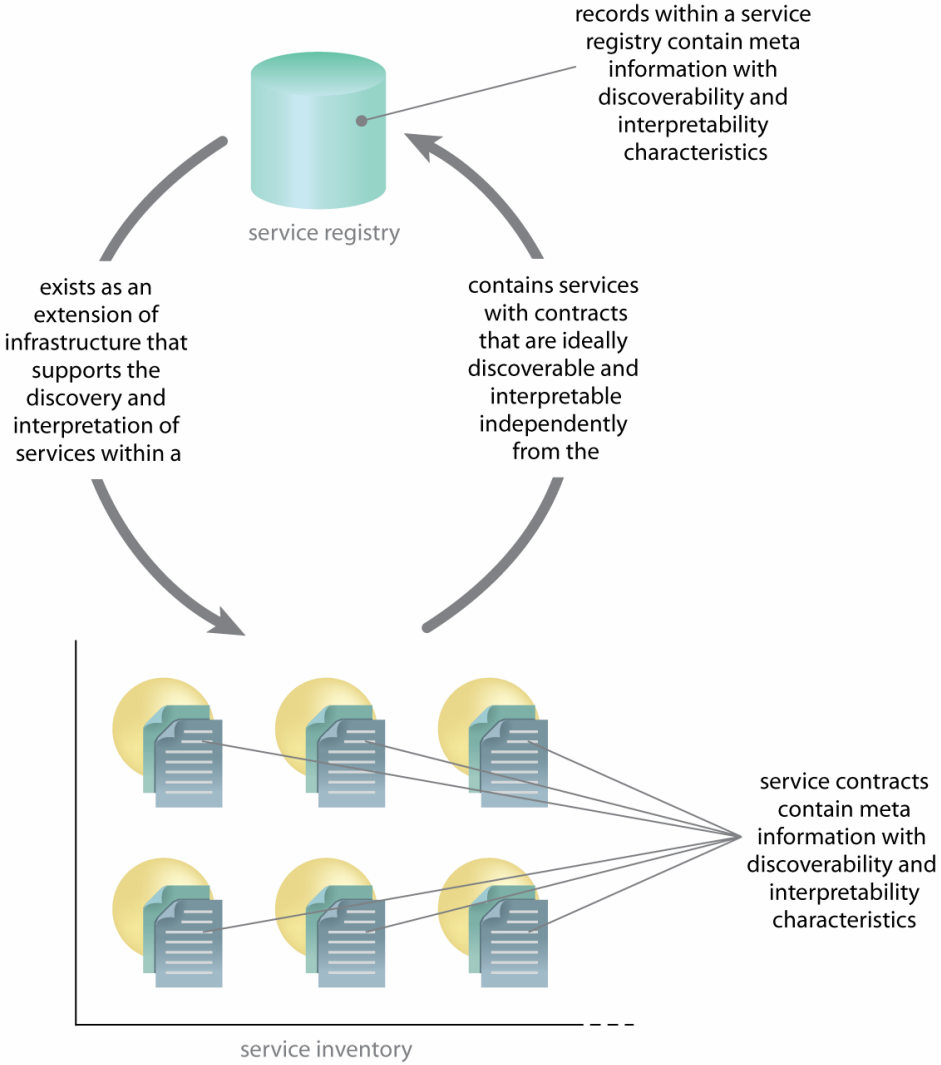
- The process of searching for and finding solution logic within a specified environment is referred to as discovery.
- For something within the enterprise to be discoverable, it needs to be equipped with meta information that will allow it to be included within the scope of discovery searches.
- Something that can adequately be discovered is considered to have a measure of discoverability.
- Discoverability information is essentially a combination of the content in a service contract and meta data in the corresponding registry record.

Service Discoverability

- Once located by a human, it is important that the purpose and capabilities of what was discovered be clearly understood.
- This level of clarity or “communications quality” is referred to as interpretability.
- The steps involved by a human to evaluate the query results of a discovery process and to then choose a service as being capable of fulfilling desired automation requirements is part of the interpretation process.

This principle is focused on both the discoverability and interpretability of service meta information.

Essentially, it comes down to the communications quality of the service contract and registry profile.



Service Composability

"Services are effective composition participants, regardless of the size and complexity of the composition."

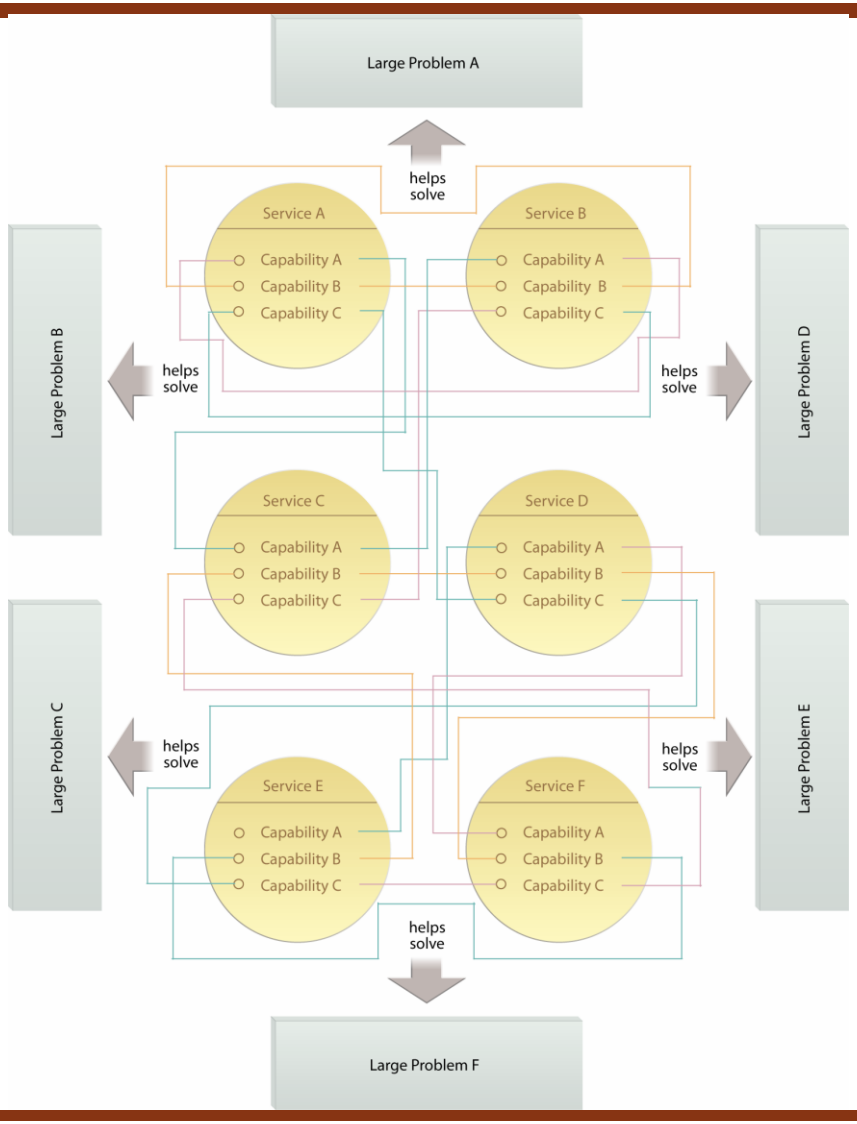
Service Composability

- The ability to effectively compose services is a critical requirement for achieving some of the fundamental goals of service-oriented computing.

- Complex service compositions place significant demands on service design.

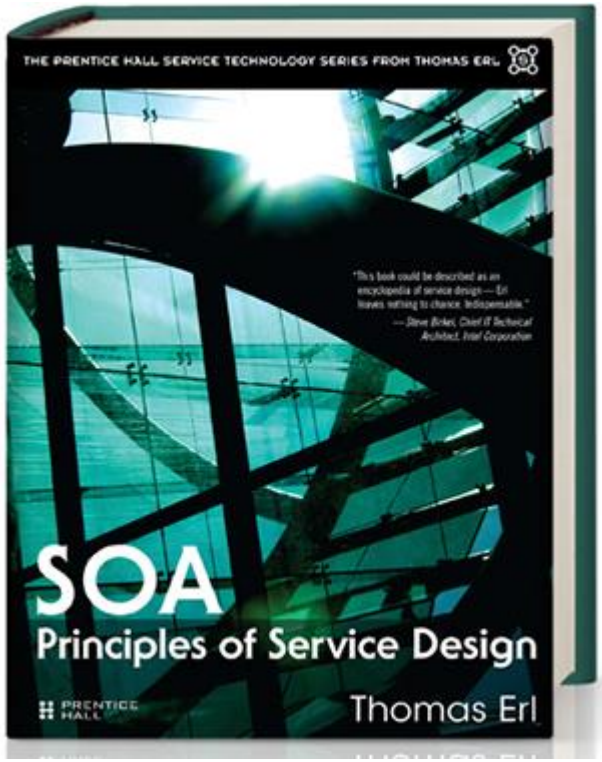
- This principle is dedicated to making services effective composition members, regardless of whether they need to be immediately enlisted in a composition.

The ultimate result of applying service-orientation is a collection of agnostic services with capabilities that can be repeatedly composed to help solve other large problems.



For more information about service-orientation design principles:

T. Erl, SOA Principles of Service Design



Or

http://serviceorientation.com/serviceorientation/index

Service Inventory Design Patterns

The following patterns are related to the definition of a service-oriented architectural model with an emphasis on service inventory architecture:

- Enterprise Inventory (116)
- Domain Inventory (123)
- Service Normalization (131)
- Logic Centralization (136)
- Schema Centralization (200)
- Service Layers (143)

Service Definition Design Patterns

The service definition process is carried out by sequentially applying the following patterns:

- Service Encapsulation (305)
- Agnostic Context (312)
- Non-Agnostic Context (319)
- Agnostic Capability (324)
- Utility Abstraction (168)
- Entity Abstraction (175)
- Process Abstraction (182)

Note that each pattern essentially corresponds to a foundational step that is part of a service definition process.

Service Architecture Design Patterns

The following patterns are primarily related to the definition of service architecture:

- Decoupled Contract (401)
- Contract Centralization (409)
- Official Endpoint (711)
- Service Façade (333)
- Concurrent Contracts (421)
- Redundant Implementation (345)
- Service Data Replication (350)
- Legacy Wrapper (441)

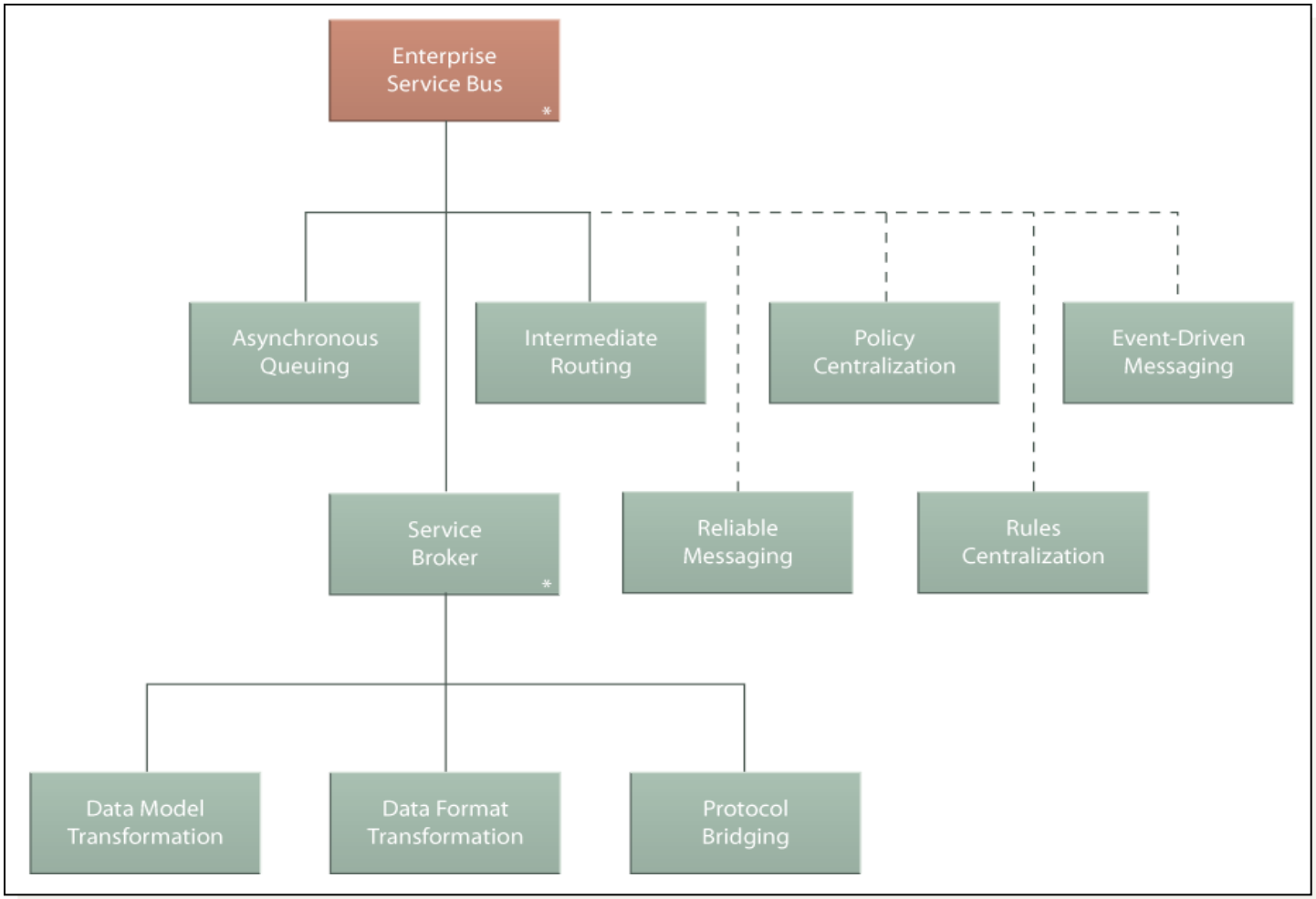
Service Composition Design Patterns

The following are fundamental patterns primarily related to the definition of service composition architecture:

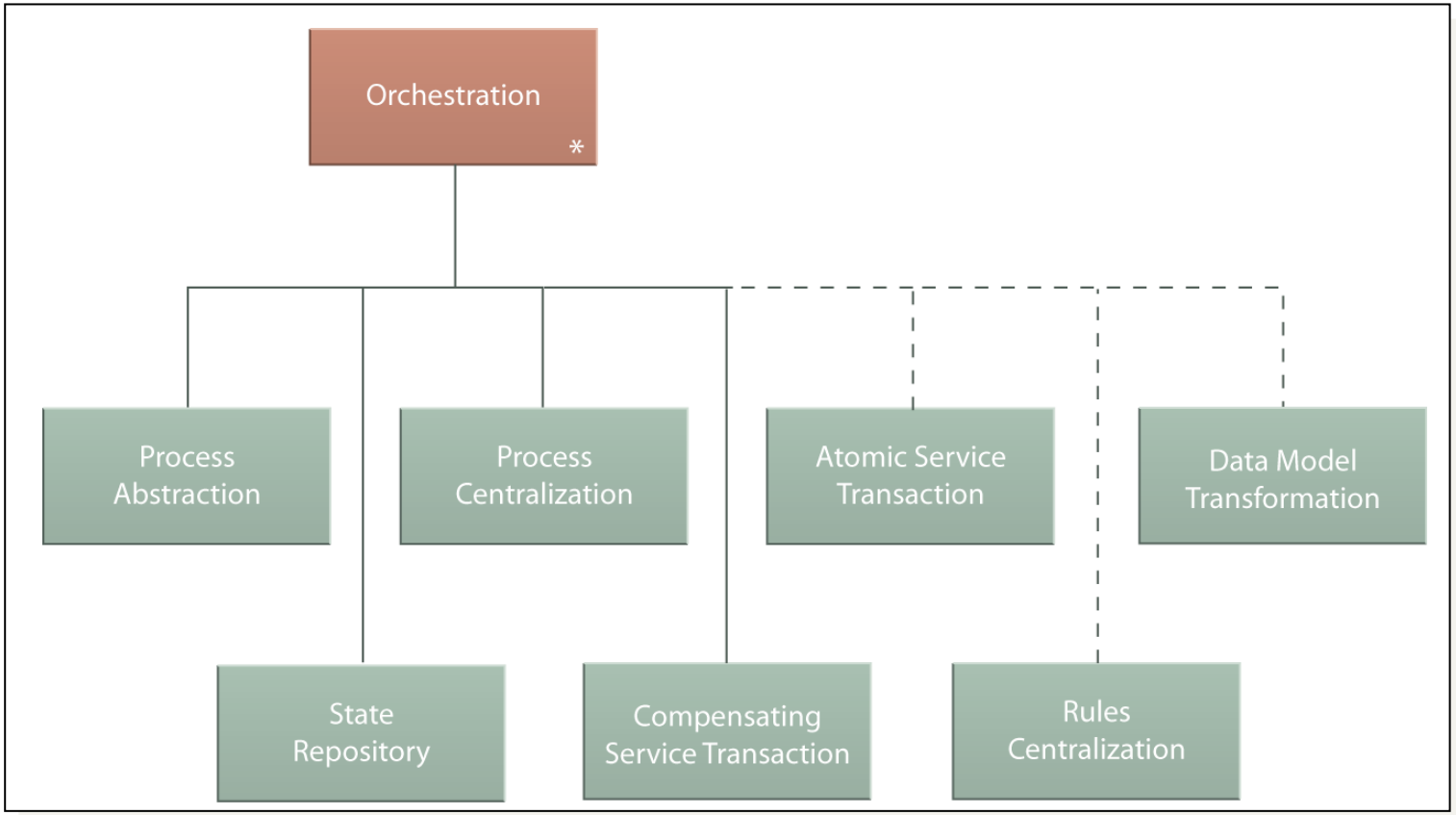
- Messaging Metadata (538)
- Service Agent (543)

Note that several additional, more specialized service composition-related patterns are covered in the upcoming Enterprise Service Bus (ESB) Patterns and Orchestration Patterns sections.

Enterprise Service Bus (704) Compound Pattern



Orchestration (701) Compound Pattern

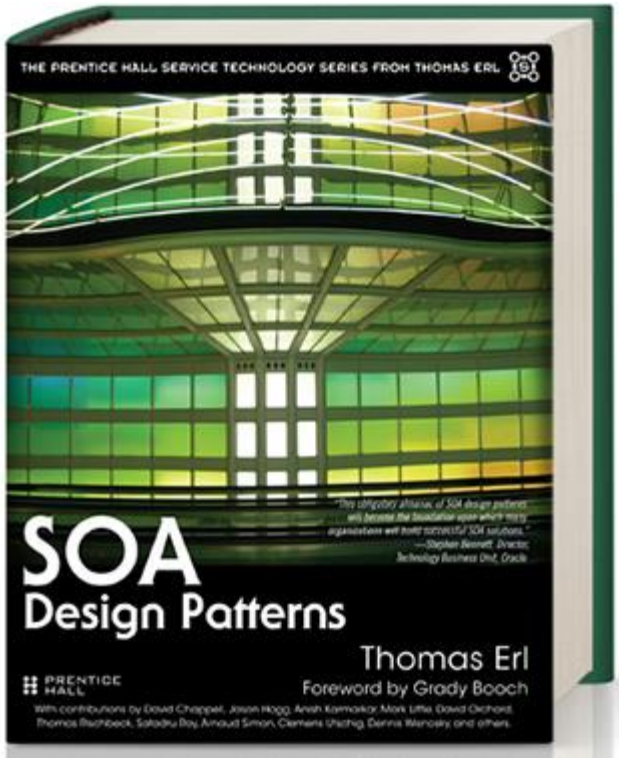


For more information
about SOA
design patterns:

T. Erl, SOA Design Patterns

Or

http://www.soapatterns.org



Part I: Fundamental SOA

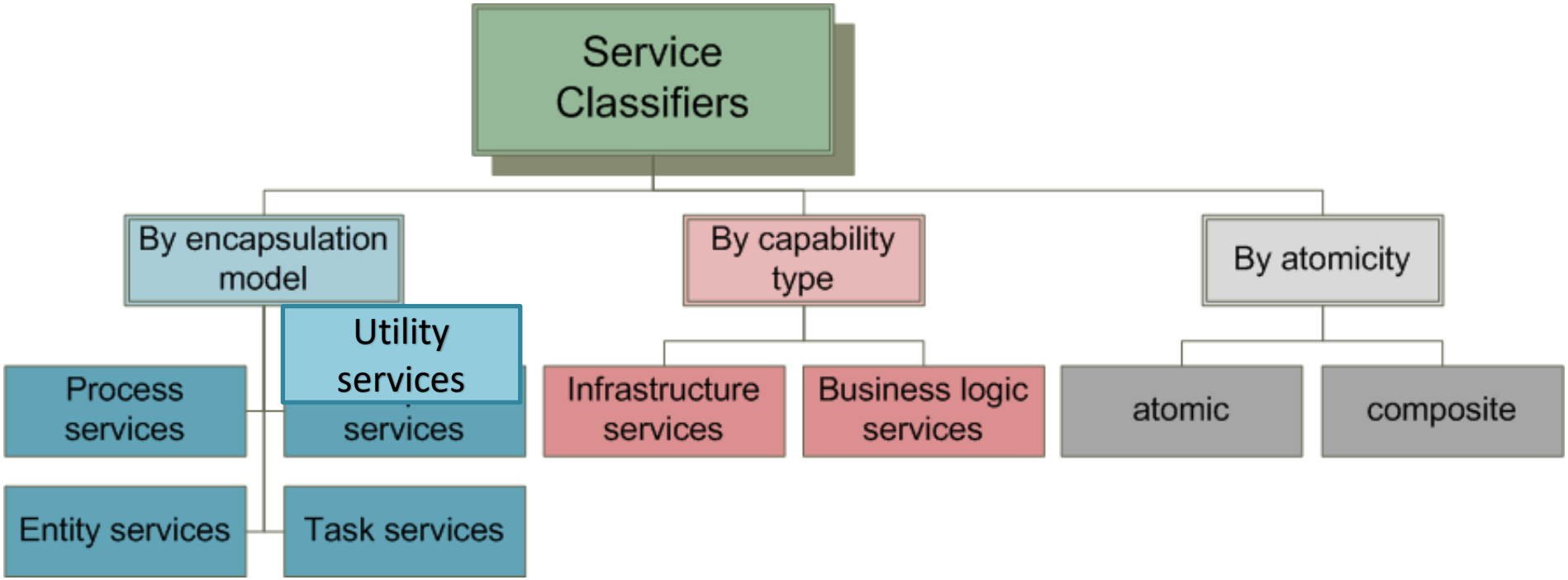
1 Introduction to SOA
2 SOA Design Principles and Patterns
3 SOA Development Methodology
4 SOA Platform Architecture

Service Modeling

Service Modeling Answers Questions

- Are we building the right services?
- How do we enable people to use the services?
- How should we build this service?
- How do we measure the value of a service?

Service Models: Examples of Service Classification



Logic Types and Service Models

	Business Logic	Utility Logic	Agnostic Logic	Non-Agnostic Logic
Utility Service		X	X	
Entity Service	X		X	
Task Service	X			X

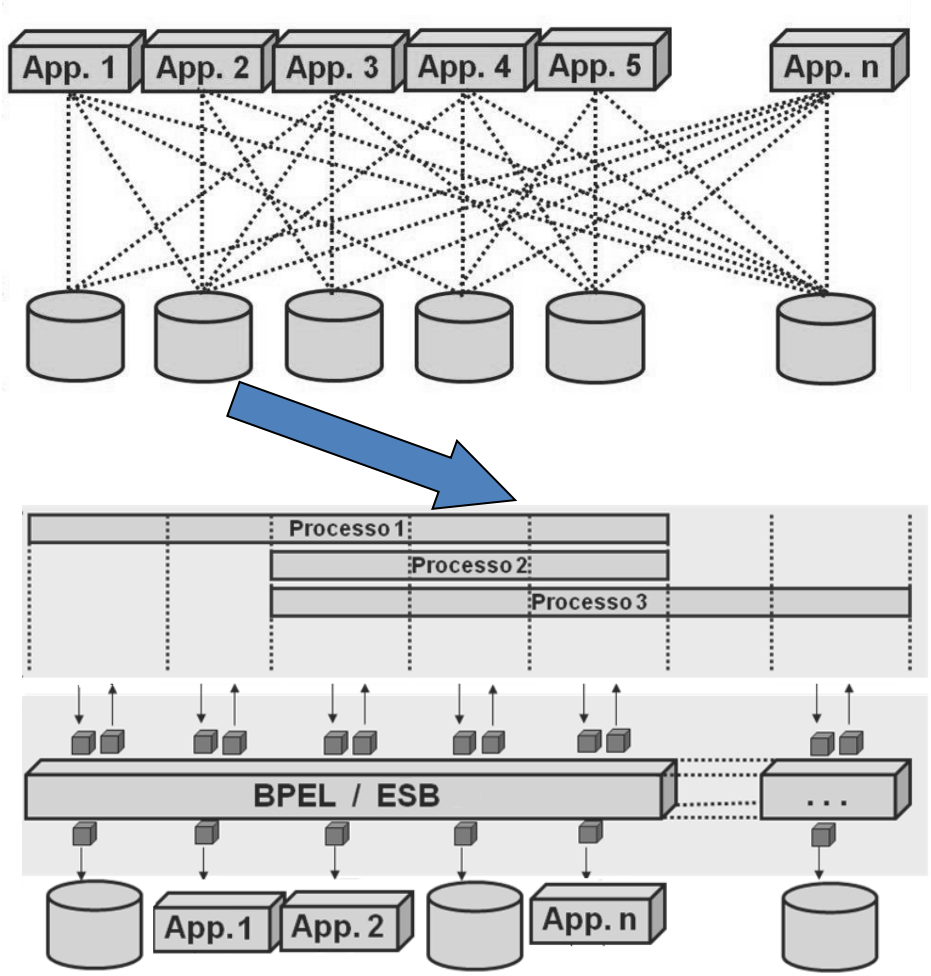
SOA Platform (aka SOA Middleware)

BPM integration
- round-trip process definition to process automation
- fast development
- computer-aided development
- business - IT alignment

Mediation technology
- communication infrastructure
- message framework
- support to common composition patterns: transform., events, security, monitoring, scaling…

System integration framework:
- all-in-one system
- legacy systems management
- standardization
- documentation

Software & IT governance:
- improved life-cycle management
- software governance
- reuse



REST Services

- REST (Representational State Transfer) originated in a Ph.D. dissertation by Roy Fielding in 2000.

- REST services do not have individual service contracts, but instead share a uniform contract via the standard HTTP protocol.

- REST services communicate with each other using HTTP methods, such as GET, PUT, POST, DELETE, HEAD, OPTIONS

- However, each REST service can be designed to support only a subset of available HTTP methods.

REST Services

- REST services leverage the underlying architecture of the World Wide Web.

- REST services enable addressability to resources via URLs and URIs.

- When a REST service is invoked, the URL or URI indicates what resource is to be accessed and what HTTP method is to be used to process the resource.

- A solution comprised of REST services is viewed as a network of linked resources.

REST Services

- When REST services exchange data, HTTP headers can be used to carry meta-information, such as authentication tokens, response codes, error information.

- Resources can be represented using different formats and protocols, such as XML, POX (Plain Old XML), or JSON (JavaScript Object Notation).

- Data exchanged by REST services can be defined using XML Schema.